Javascript Functional Programming - Assessment Brief

Attached Files
COMP2140_A1_FunctionalJavascript_Rubric_V1.docx (78.44 KB)
proxy_windows_v4.zip (6.911 MB)

Due Date	Grade Weighting
3pm AEST Due Date: Friday, 30 August 2024 (Week 6)	30% of Course Grade

Overview

In the *JavaScript Functional Programming* assessment, you'll build a terminal-based, Node.js app that demonstrates your knowledge & application of functional programming in JavaScript. This assessment is designed to extend your skills in JavaScript & programming fundamentals, through the prerequisite courses, DECO1400 (JavaScript) & CSSE1001 (Python).

To achieve this, you'll submit towards the following deliverable:

- *Final JS App*

Learning Objectives

After successfully completing this assessment, you should be able to in part:

- 1. Apply an intermediate knowledge of functional programming with vanilla JavaScript.
- 4. Communicate with web & device APIs that are shared across web/native apps.
- 6. Evaluate the design, implementation & architecture for delivering complex, cross-platform web/native apps.
- 7. Develop customised testing regimes for evaluating design, code & security of cross-platform web/native apps.

What Am I Building?

For this assessment, you'll be building a fully-functional TransLink data parser that provides a summary of all Route stops. This is similar to what you might see when you lookup a Route (e.g. 169 or 209), but it will run as a Node.js app in the terminal.

With this app, you're asked to demonstrate all of what you've learned & practiced in Lectures & Contacts from Week 1 to 5. This includes, but is not limited to:

- Techniques for functional programming in JavaScript:
  - A fully declarative programming style (minimising imperative programming)
  - Nested functions, function composition and function chaining
  - Immutability and transformation of data within the app
  - Use of modules and re-usability
  - Asynchronous programming through fetching of local & live data
  - Consideration of dependancies management.
- Techniques for testing applications:
  - Building comprehensive test cases to test modules and functionality
- An easy to understand JavaScript code style:
  - Readable with a considered program structure, variable and function names
  - Neatness with tabbing and whitespace
  - Clear comments in plain English that provide a meaningful description of the code, such as in JSDoc style.
  - Use of modern JavaScript shorthand where appropriate to use
  - Consideration of good practice via popular style guides.

Where Do I Start?

1. Download & extract **SEQ_GTFS.zip**

- This ZIP file contains static data files that you'll read, parse & combine as you run your JavaScript app with information about the TransLink network in South East Queensland.
- You should save these static data files in your project folder and read them from there.
- Part of your challenge is to decipher what the static data files mean and how you can use them (i.e. link related columns to extract the data you need). GTFS is a specification and more info can be found here: https://translink.com.au/about-translink/open-data

2. Run the provided server application to start up an API proxy for **trip_update.json** and **vehicle_positions.json** to be enabled via the following local URLs:

URL	Description
http://127.0.0.1:5343/gtfs/seq/trip_updates.json	After running the server application, this will load a JSON version of SEQ Trip Updates (originally in Protobuf format).
http://127.0.0.1:5343/gtfs/seq/vehicle_positions.json	After running the server application, this will load a JSON version of SEQ Vehicle Positions (originally in Protobuf format).
http://127.0.0.1:5343/gtfs/seq/alerts.json	After running the server application, this will load a JSON version of SEQ Alerts (originally in Protobuf format).

   - The ZIP file below contains binaries to be run via the terminal (for macOS and Linux) & an executable to run (for Windows). When successfully run, the above local URLs will be enabled.
     proxy_server_v3.zip
   - This ZIP file has been updated to include v2 of the API proxy server binaries, which add a *-static* command line option to help with developing your app after hours. When the
     -static option has been appended, the API will return data from a point in time.
   - If you're attempting to run the binaries in the terminal, you will need to set execute permissions first. For example:
     chmod +x assign1_server_macos_m1 for Mac and
     chmod +x assign1_server_linux for linux.
   - This data is proxied in real-time from the "TransLink GTFS Real-Time Feed" API into JSON format (from Protobuf, a binary format). This means you're dealing with near-live, cached data that needs filtering for Route stops at a specific day/time.
   - You should cache this real-time data locally by saving the JSON files into your project folder and read them from there. Caching will improve performance so that the API is not called so frequently. The caches should be regularly flushed (e.g. every 5 mins) in order to keep your caches up to date.
   - Again, part of your challenge is to decipher what the JSON returned means and how you can use it.

3. Create a project folder with the following subfolders:

File/Folder Name	Description
static-data/	This is where you could store the static data files from SEQ_GTFS.zip.
cached-data/	This is where you could store locally cached versions of the real-time data from trip_updates.json and vehicle_positions.json.
translink-parser.js	This is where the main logic of your app could go.

4. Before coding, we recommend that you plan out how your app may be structured. This could be achieved by producing a flow chart that outlines the functions & data communication. **Please include your Planning Document in your submission.**

5. Design and implement a module that will serve as a dataframe (i.e. CSV) processing library. It needs to include methods to load a CSV file into a dataframe, join two CSV files on a common field, select specific columns from a dataframe, return distinct values from a column and filter columns of a dataframe using criteria involving comparison operators. Be strategic and only include filtering options that are required by the application.

6. Use your dataframe module to provide the re-usable functionality you need to process the GTFS dataset for your Route planner.

7. Develop a set of comprehensive tests using Jest.

calendar dates: exception_date (done)
calendar: weekday (done)
stoptime: date range (done)
stop: start stop || end stop (done)
trips: direction (done)
stop_times: arrival_time(10 mins)

How Should the App Function?

1. When your app is run via Node.js in the terminal, show the following message:

   Welcome to the South East Queensland Route Planner!

2. The user will then be prompted for the following input:

Prompt	Expected Values
What Bus Route would you like to take?	The user should be able to enter a Route number: i.e. 66, 40 Error message for a number that does not match a Route

Prompt	Expected Values
	"Please enter a valid bus route." If a valid Route is entered, display a list of Stops with a numeric id. e.g. 1. UQ Lakes stop D 2. Dutton Park Place 3. Boggo Road station, platform 5 4. ....
What is your start and end stop on the route?	The user must enter data in this format: e.g. 1 - 2 (This would be to travel from 1 - 1. UQ Lakes stop D to 2 - Boggo Road station, platform 5) Error message for the user not following the format or entering non-numeric or out of range values "Please follow the format and enter a valid number for the stop"
What date will you take the route?	Year, month & day in https://tc39.es/ecma262/#sec-date-time-string-format (YYYY-MM-DD) Error message for incorrect format: "Incorrect date format. Please use YYYY-MM-DD"
What time will you leave?	Hour & minutes in 24 hour time in https://tc39.es/ecma262/#sec-date-time-string-format (HH:mm) Error message for incorrect format: "Incorrect time format. Please use HH:mm"

When testing your app, we recommend using the current date & time.

3. After the prompts are validated against the expected values, parse & output the following using **console.table()** for each result that is scheduled to arrive less than 10 minutes from the current time:
    - The short name for the route (Table column name: Route Short Name);
    - The long name for the route (Table column name: Route Long Name);

- The service ID for the trip (Table column name: Service ID);
- The head sign for the trip (otherwise known as destination sign) (Table column name: Heading Sign);
- The scheduled arrival time of the vehicle at the starting stop (Table column name: Scheduled Arrival Time);
- The live arrival time of the vehicle at the starting stop (Table column name: Live Arrival Time);
- The live geographic position of vehicle (Table column name: Live Position).
- Estimated time to destination stop (Table column name: Estimated Travel Time). The calculation of travel time does not need to take into account any live data.

  Note: If the route starts at stop, there may not be an arrival time available. In this edge case, use the departure time as a suitable equivalent.

4. The user should be able to restart the app by being prompted for further input:

Prompt	Expected Values
Would you like to search again?	y, yes, n, no (case-insensitive) Error message for incorrect expected value: "Please enter a valid option."

5. After the prompt is validated against the expected values, either restart the app or return this message:

   Thanks for using the Route tracker!

Other Specifications:

- In addition to modules provided in the Node.js core (such as the "fs" module), you may only use the following individual/sets of NPM modules as dependancies in your app:
  - The synchronous module "prompt-sync" or the asynchronous module "prompt".
  - The module "node-fetch" for enabling Fetch API in Node.js.
  - Modules for parsing CSV files within the "CSV for Node.js" suite.
- You are not allowed to use other libraries especially Dataframe or Data Science libraries or load the CSV files into a database (e.g., MySQL or SQLLite) and use SQL to filter the data.
- Ensure you double-check that any text content within your app is checked for spelling, grammar, punctuation & naming.
- You can use Typescript (if you already know Typescript) but it is not mandatory.

- **Ensure you include your planning document in your submission.**

Additional Questions:

If you have any questions about this assessment brief, you're welcome to post them on the course Ed Discussion and we'll get back to you soon.

Final JS App

In **Week 6**, you'll submit your *Final JS App* as a digital submission to Gradescope.

⏰ You are to submit a ZIP file of your *Final JS App* to Gradescope (linked via the *JavaScript Functional Programming* assessment on Blackboard) by **3pm AEST on Friday, 30 August 2024** (Week 6).

Marking Criteria:

The marking Rubric is attached.

A Message About Plagiarism:

⚠️ **Plagiarism is considered a serious offence at UQ.** Failure to declare the distinction between your work and the work of others will result in academic misconduct proceedings.

- All design & code for this assessment must be entirely your own, except for pre-approved snippets or assets provided by the assessment brief. Treat what you're producing here as a "trade secret".
- If you're inspired by design or code from online tutorials or any other external source, ensure that they are completely recreated in your own style. Ensure you reference any inspirations for academic purposes (using APA/IEEE referencing styles).
- You must first seek approval from teaching staff to use anything directly from external sources. Check with teaching staff if you need further clarifications about this.

Content & Technical Specifications:

- In your submission, your ZIP file should contain:

    - A parent folder named to the format below. Your ZIP file should be the same name.

        First & Last Name (Student Number)

    - Your project folder copied inside the parent folder, containing your JavaScript file(s), the static data files and cached real-time data.

        💡 Note, due to Gradescope's per-file size limit of <100 MB, you can skip uploading the static data file **stop_times.txt**. The autograder will automatically re-instate the file so that your submission is complete.
- As your submission will contain primarily text & code, we expect that your ZIP file will be no larger than 500 MB in size. If it is larger than 500 MB, your real-time data caches may need to be pruned.
- You can submit more than one attempt to Gradescope if you make a mistake. Just ensure you submit your final attempt before the due time.
- Uploading to Gradescope may take time, especially with a large submission. If you have any problems submitting your ZIP file to Gradescope, email Aneesha (the course coordinator) ASAP before the due date/time.
- Your latest attempt will be downloaded, marked & checked for similarity by teaching staff.
- **Please note that the use of Generative AI such as ChatGPT, Github Copilot, Anthrolpic Claude are allowed in this assessment item. Please reference your usage of GenAI.**

Gradescope Autograder

The Autograder will only check that your Node.js app is able to be installed without any issues. Additional tests will be performed when grading your submission.

Assessment Brief Change Log

Update	Date
Added 2 new FAQ's	28 Jul 2024
Moved FAQ out of the Brief and onto the assessment page.	8 Aug 2024

Updated the proxy server to version 4 with static files for the latest SEQ_GTFS.zip download.	8 Aug 2024
Restricted testing to only 2 Bus Routes (i.e. 66 and 40)	8 Aug 2024